

Lab Manual for Data Structure and Algorithm

Week06_(LAB-05)

MS. MUNIBA KHAN

Linked list Insertion, Traversal, Searching, Deletion, Sort, Search, Reverse, Tail Pointer

Lab 5	
Contents:	Linked list Insertion, Traversal, Searching, Deletion, Sort, Search, Reverse, Tail Pointers

Linked List:

A **Linked List** is a linear dynamic data structure consisting of nodes.

Each node contains:

- data – the value stored
- next – pointer to the next node

Structure of a Node

Each node has two main parts:

- **Data** – stores the value (e.g., integer, float, or any type).
- **Next** – stores the **address of the next node** in the list.
- The last node points to NULL indicating the **end of the list**.
- The first node is called the **head**.
- Optionally, we can have a **tail pointer** pointing to the last node for faster insertion at the end.

Advantages over arrays:

- Dynamic size
- Easy insertion & deletion

Types:

- Singly Linked List
- Doubly Linked List
- Circular Linked List

1. Insertion

Insertion means adding a new node into the linked list.

There are three common ways:

a. Insert at Beginning

- A new node becomes the first node.
- Its next pointer is linked to the previous first node.
- Time Complexity: $O(1)$ (very fast)

b. Insert at Last

- Traverses the linked list until the last node.
- Adds the new node at the end.
- Time Complexity: $O(n)$

c. Insert at a Given Position

- Traverse to the desired position.
- Adjust pointers to insert the new node.
- Time Complexity: $O(n)$

2. Traversal

Traversal means visiting each node of the linked list one by one.

How it works:

- Start from the head (or start) pointer.
- Continue moving using `node = node->next`.
- Stop when `node == NULL`.

Used for:

Displaying elements

- Searching
- Counting nodes

Time Complexity: $O(n)$

3. Searching

Searching means finding whether a particular value exists in the list.

How it works:

- Traverse through the entire list.
- Compare each node's data with the target value.
- If found, return its position.

Linear Search → Time Complexity: $O(n)$

(Because a linked list cannot perform binary search)

4. Deletion

Deletion means removing a node from the linked list.

Three common cases:

a. Delete at Beginning

- Move the head pointer to the second node.
- Free memory of the first node.

b. Delete at a Specific Position

- Traverse to the previous node of the node to delete.
- Adjust pointers: `prev->next = temp->next`
- Free the node.

c. Delete at Last

- Traverse until second-last node.
- Set its next pointer to NULL.
- Free last node.

Time Complexity: $O(n)$

5. Sorting

Sorting means arranging the linked list in ascending order.

Because a linked list has no random access, sorting is usually done by:

Bubble Sort or Selection Sort (Node Swapping or Data Swapping)

Procedure (data swapping method):

- Compare each node with the rest.
- Swap values if out of order.
- Continue until list becomes sorted.

Time Complexity: $O(n^2)$

6. Update

Updating means replacing the value stored at a specific node.

Steps:

1. Traverse the list until the specified position.
2. Replace the node's data with the new value.

Time Complexity: $O(n)$

7. Reverse

Reversing a linked list means the start becomes the last, and the last becomes the first.

How it works (using three pointers):

- ptr1 → previous node
- ptr2 → current node
- ptr3 → next node

While traversing, every next pointer is reversed.

Time Complexity: $O(n)$

Space Complexity: $O(1)$ (in-place reverse)

8. Tail Pointer

A tail pointer stores the address of the last node of the linked list.

It allows fast insertion at the end.

Benefits:

- Insert-at-last becomes $O(1)$ instead of $O(n)$
- No need to traverse the entire list

Structure:

head → first node

tail → last node (tail->next = NULL)

If the list is empty:

head = tail = NULL

When inserting:

- If empty → new node becomes both head and tail
- Otherwise → tail->next = newNode, update tail

Lab Tasks:

Write a program to create a structure of a node, create a class Linked List. Implement all operations of a linked list as member function of this class.

- create_node(int);
- insert_begin();
- insert_pos();
- insert_last();
- delete_pos();
- sort();
- search();
- update();
- reverse();
- display();

Solution

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

class LinkedList {
private:
    Node* head;
    Node* tail;

public:
    LinkedList() {
        head = NULL;
        tail = NULL;
    }

    Node* createNode(int value) {
        Node* temp = new Node;
        if (temp == NULL) {
            cout << "Memory not allocated!" << endl;
            return NULL;
        }
    }
```

```

    }
    temp->data = value;
    temp->next = NULL;
    return temp;
}

void insertBegin(int value) {
    Node* temp = createNode(value);
    if (head == NULL) {
        head = tail = temp;
    } else {
        temp->next = head;
        head = temp;
    }
    cout << "Inserted " << value << " at beginning.\n";
}

void insertLast(int value) {
    Node* temp = createNode(value);
    if (head == NULL) {
        head = tail = temp;
    } else {
        tail->next = temp;
        tail = temp;
    }
    cout << "Inserted " << value << " at end.\n";
}

void insertPos(int value, int pos) {
    Node* temp = createNode(value);
    if (pos < 1) {
        cout << "Position should be >= 1\n";
        return;
    }

    if (pos == 1) {
        temp->next = head;
        head = temp;
        if (tail == NULL) tail = temp;
        cout << "Inserted " << value << " at position " << pos << endl;
        return;
    }

    Node* current = head;
    for (int i = 1; i < pos - 1; i++) {
        if (current == NULL) {
            cout << "Position out of range\n";
            return;
        }
        current = current->next;
    }
}

```

```

    if (current == NULL) {
        cout << "Position out of range\n";
        return;
    }

    temp->next = current->next;
    current->next = temp;
    if (temp->next == NULL) tail = temp;
    cout << "Inserted " << value << " at position " << pos << endl;
}

void deletePos(int pos) {
    if (head == NULL) {
        cout << "List is empty\n";
        return;
    }

    if (pos < 1) {
        cout << "Invalid position\n";
        return;
    }

    Node* temp;
    if (pos == 1) {
        temp = head;
        head = head->next;
        if (head == NULL) tail = NULL;
        delete temp;
        cout << "Deleted node at position 1\n";
        return;
    }

    Node* current = head;
    for (int i = 1; i < pos - 1; i++) {
        if (current == NULL) {
            cout << "Position out of range\n";
            return;
        }
        current = current->next;
    }

    if (current->next == NULL) {
        cout << "Position out of range\n";
        return;
    }

    temp = current->next;
    current->next = temp->next;
    if (current->next == NULL) tail = current;
    delete temp;
    cout << "Deleted node at position " << pos << endl;
}

```

```

void updateNode(int pos, int value) {
    if (head == NULL) {
        cout << "List is empty\n";
        return;
    }

    Node* current = head;
    for (int i = 1; i < pos; i++) {
        if (current == NULL) {
            cout << "Position out of range\n";
            return;
        }
        current = current->next;
    }

    if (current == NULL) {
        cout << "Position out of range\n";
        return;
    }

    current->data = value;
    cout << "Updated node at position " << pos << " to " << value << endl;
}

void searchElement(int value) {
    Node* current = head;
    int pos = 1;
    bool found = false;
    while (current != NULL) {
        if (current->data == value) {
            cout << "Element " << value << " found at position " << pos << endl;
            found = true;
            break;
        }
        current = current->next;
        pos++;
    }
    if (!found) cout << "Element " << value << " not found\n";
}

void display() {
    if (head == NULL) {
        cout << "List is empty\n";
        return;
    }
    Node* current = head;
    cout << "Linked List: ";
    while (current != NULL) {
        cout << current->data << " -> ";
        current = current->next;
    }
}

```

```

        cout << "NULL\n";
    }

    void reverse() {
        Node* prev = NULL;
        Node* current = head;
        Node* next = NULL;
        tail = head;
        while (current != NULL) {
            next = current->next;
            current->next = prev;
            prev = current;
            current = next;
        }
        head = prev;
        cout << "Linked List reversed\n";
    }

    void sortList() {
        if (head == NULL) {
            cout << "List is empty\n";
            return;
        }

        for (Node* i = head; i != NULL; i = i->next) {
            for (Node* j = i->next; j != NULL; j = j->next) {
                if (i->data > j->data) {
                    int temp = i->data;
                    i->data = j->data;
                    j->data = temp;
                }
            }
        }
        cout << "Linked List sorted in ascending order\n";
    }
};

int main() {
    LinkedList list;
    int choice, value, pos;

    do {
        cout << "\n\n--- Linked List Operations ---\n";
        cout << "1. Insert at Beginning\n";
        cout << "2. Insert at End\n";
        cout << "3. Insert at Position\n";
        cout << "4. Delete at Position\n";
        cout << "5. Update Node\n";
        cout << "6. Search Element\n";
        cout << "7. Display List\n";
        cout << "8. Reverse List\n";
        cout << "9. Sort List\n";
    }

```

```

        cout << "10. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter value: "; cin >> value;
                list.insertBegin(value); break;
            case 2:
                cout << "Enter value: "; cin >> value;
                list.insertLast(value); break;
            case 3:
                cout << "Enter value: "; cin >> value;
                cout << "Enter position: "; cin >> pos;
                list.insertPos(value, pos); break;
            case 4:
                cout << "Enter position to delete: "; cin >> pos;
                list.deletePos(pos); break;
            case 5:
                cout << "Enter position to update: "; cin >> pos;
                cout << "Enter new value: "; cin >> value;
                list.updateNode(pos, value); break;
            case 6:
                cout << "Enter element to search: "; cin >> value;
                list.searchElement(value); break;
            case 7:
                list.display(); break;
            case 8:
                list.reverse(); break;
            case 9:
                list.sortList(); break;
            case 10:
                cout << "Exiting...\n"; break;
            default:
                cout << "Invalid choice!\n";
        }
    } while (choice != 10);

    return 0;
}

```

```

3.Insert node at position
4.Sort Link List
5.Delete a Particular Node
6.Update Node Value
7.Search Element
8.Display Linked list
9.Reverse Linked list
10.Exit
Enter your choice : 7
Search element in Link List:
Enter the value to be searched: 21
Element 21 is found at position 1

-----
Operations on singly linked list
-----
1.Insert Node at beginning
2.Insert node at last
3.Insert node at position
4.Sort Link List
5.Delete a Particular Node
6.Update Node Value
7.Search Element
8.Display Linked list
9.Reverse Linked List
10.Exit
Enter your choice :

```

Lab Tasks:

Problem 1 – Linked List Insertion & Display

Task:

1. Write a program to create a **Singly Linked List**.
2. Implement functions to:
 - Insert a node at the **beginning**
 - Insert a node at the **end**
 - Insert a node at a **specific position**
3. Implement a function display() to traverse and print the linked list.

Test Case:

- Insert 10 at beginning, 20 at end, 15 at position 2.
- Expected Output: 10 -> 15 -> 20 -> NULL

Problem 2 – Searching & Deletion in Linked List

Task:

1. Implement a function searchElement(int value) to find the position of a value in the list.
2. Implement a function deletePos(int pos) to delete a node at a given position.
3. Test both functions with your linked list.

Test Case:

- Current List: 5 -> 10 -> 15 -> 20 -> NULL
- Search for 15 → Output: Element 15 found at position 3
- Delete node at position 2 → Resulting List: 5 -> 15 -> 20 -> NULL

